HaCI: Harmonic Contour Integration for Edge Detection

Siddharth Basireddy

October 14, 2026

Abstract

This paper describes HaCI (Harmonic Contour Integration), a compact, fully distributed edge detection method. Through the encoding of classical early-visual priors like divisive normalisation, and association-field facilitation and suppression, the method exposes a network made up of a small set of 159 learnable parameters. The method is composed of four staged operations on an image: the construction of a per-pixel contrast signal and its projection onto circular harmonics (L0), pooling of the harmonics over overlapping cells into von Mises orientation bins (L1), refinement of the signal via facilitation and suppression (Seed) and the eventual projection of the pooled signal back onto the pixel level (renderer). The renderer back projection is then passed through non-max suppression to retrieve clean edges.

Keywords. Contour Integration, Harmonic Projection, Divisive Normalisation, Filtered Back-Projection, Edge Detection.

1 Introduction

One of the primary functions of the early visual cortex is the transformation of retinal input into global contours. As the cortex operates in a highly parallelized fashion, low-level signals are normalized across the visual grid and passed through layers of increasing radius that accumulate evidence across scale. These low-level signals are subjected to cortical transformations such as non-linear saturation [1], oriented filtering [2], divisive normalization [3, 4], lateral facilitation and inhibition [5, 6] and figure-ground polarity [7]. At higher levels, contour readouts are sharpened by collinear and co-circular support and normalised against the surrounding activity before being deposited onto a coherent boundary map.

Analogs to the cortical architecture for computer vision require transforming sparse, irregular mechanisms into dense, synchronous operations suitable for array programming models and parallel processing hardware. Deep convolutional and transformer edge detectors have become the dominant approach [14, 15, 16, 17, 18, 19], recovering contour structure implicitly through millions of learned weights that carry structural priors from the training data alone. The method introduced in this paper revisits older pipelines built from explicit structural priors, like Canny [10] and gPb [11], which construct a pixel-level signal and improve it through an interpretable sequence of stages, but hold gain, scale and anisotropy fixed at design time. We deviate from these gradient-based methods, opting for harmonic projection and encoding cortex-like mechanisms as the core heuristic of a compact, fully differentiable four-stage graph, that extracts thin edges from an RGB image while exposing only 159 learnable scalars to data.

2 Method

The pipeline operates in four stages; L0, L1, seed and renderer, each with a fixed radius of interaction.

2.1 L0: Harmonic Projection

Let $I : \Omega \to \mathbb{R}^3$ be the input image in sRGB, scaled to $[0, 1]$; no linearisation is applied. Before L0 the image is reflect-padded at the bottom and right so that the L1 cell grid (section 2.2) covers it exactly, and Ω denotes this padded domain; the boundary map is cropped back to the original image $\Omega_0 \subseteq \Omega$ at the end. The eight-neighbour offsets $\delta_k = (\delta_{y,k}, \delta_{x,k})$, $k = 1, \ldots, 8$, in image (row, column) order, have bearings $\phi_k = \operatorname{atan2}(-\delta_{y,k}, \delta_{x,k}) \in \{0, \pi/4, \ldots, 7\pi/4\}$, measured anticlockwise from the $+x$ axis with y pointing up (image rows run downwards, hence the sign). Luminance and chrominance are routed through separate channels of a learned spectral metric, divisively normalised against their own energy and a spatial pool of local contrast, and projected onto circular harmonics to produce per-pixel complex moments $z_1, z_2 \in \mathbb{C}$.

Spectral RGB metric: Let $L = (1, 1, 1)/\sqrt{3}$ be the (fixed) achromatic axis and let (B_1, B_2) be a fixed orthonormal basis of $L^\perp$ in $\mathbb{R}^3$. The metric tensor on RGB differences is

$$M = W^\top W = w_\ell^2 \, LL^\top + s_1^2 \, b_1 b_1^\top + s_2^2 \, b_2 b_2^\top, \tag{1}$$

where (b_1, b_2) is the in-plane rotation of (B_1, B_2) by a learned angle θ_{chr}:

$$b_1 = \cos\theta_{\text{chr}} \, B_1 + \sin\theta_{\text{chr}} \, B_2, \qquad b_2 = -\sin\theta_{\text{chr}} \, B_1 + \cos\theta_{\text{chr}} \, B_2. \tag{2}$$

The four learned scalars are $(w_\ell, s_1, s_2, \theta_{\text{chr}})$ with $w_\ell, s_1, s_2 \geq 0$ via softplus. Applying the rotation to the basis vectors (rather than as a left multiplier on the row-stack of W) is required: the latter would make θ_{chr} cancel from $W^\top W$ by orthogonality and become pure gauge. The projected image is $u(p) = WI(p) \in \mathbb{R}^3$, where the rows of W are $w_\ell L$, $s_1 b_1$ and $s_2 b_2$.

JPEG notch: Each projected channel u_j is filtered by a learnable separable notch, $\tilde{u}_j = h_N * u_j$, applied along rows and then columns with replicate padding. The 1D frequency response is real and even,

$$\hat{h}_N(\omega) = 1 - d \exp\left(-\frac{(|\omega| - \omega_n)^2}{2\sigma_n^2} \right), \tag{3}$$

and the kernel of half-width H_N is synthesised from it by a cosine sum over the $L_N = 2H_N + 1$ sampled frequencies $\omega_m = m/L_N$:

$$h_N[n] = \frac{1}{L_N} \sum_{m=-H_N}^{H_N} \hat{h}_N(\omega_m) \cos(2\pi\omega_m n), \qquad n \in \{-H_N, \ldots, H_N\}. \tag{4}$$

The three learnable scalars are $\omega_n = \frac{1}{2}\sigma(\cdot) \in (0, \frac{1}{2})$ cycles/px, $\sigma_n = \text{softplus}(\cdot)$ and $d = \sigma(\cdot) \in (0, 1)$. They are initialised as a notch at the 8-px JPEG block frequency, $\omega_n = 1/8$, $\sigma_n = 1/32$, $d = 0.8$; the learned response is discussed in section 4.

Directional contrasts: For each pixel p and offset δ_k, the squared luminance and chrominance contrasts are taken on the filtered projection,

$$m_k^{\text{lum}}(p)^2 = \big(\tilde{u}_1(p) - \tilde{u}_1(p - \delta_k)\big)^2, \qquad m_k^{\text{chr}}(p)^2 = \sum_{j=2}^{3} \big(\tilde{u}_j(p) - \tilde{u}_j(p - \delta_k)\big)^2, \tag{5}$$

and set to zero where $p - \delta_k$ falls outside Ω. Without the notch ($\tilde{u} = u$) these are, by linearity, $w_\ell^2 (L \cdot \Delta c_k)^2$ and $s_1^2 (b_1 \cdot \Delta c_k)^2 + s_2^2 (b_2 \cdot \Delta c_k)^2$ with $\Delta c_k(p) = I(p) - I(p - \delta_k)$: the squared norms of Δc_k under the luminance and chrominance blocks of M.

Contrast gain control: Each channel $\chi \in \{\mathrm{lum}, \mathrm{chr}\}$ is passed through a rational saturating nonlinearity [1] whose denominator also carries the contrast energy pooled over all directions and over a spatial neighbourhood [3]:

$$h_k^\chi(p) = \frac{m_k^\chi(p)^2}{\eta_\chi^2 + m_k^\chi(p)^2 + \lambda_0\, Q^\chi(p)}, \qquad Q^\chi = G_{\sigma_r} * \sum_{k'=1}^{8} \big(m_{k'}^\chi\big)^2, \tag{6}$$

with $h_k(p) = h_k^{\mathrm{lum}}(p) + h_k^{\mathrm{chr}}(p)$. G_{σ_r} is a normalised separable Gaussian truncated at radius r_Q (reflect padding). The pool weight λ_0 and width σ_r are learned through softplus; at $\lambda_0 = 0$, eq. (6) reduces to a per-direction Naka–Rushton squash. The half-saturation constants $\eta_{\mathrm{lum}}, \eta_{\mathrm{chr}}$ are fixed (table 2): only m/η enters, and the metric gains w_ℓ, s_1, s_2 already set that scale.

Harmonic projection: The eight values $\{h_k(p)\}_{k=1}^8$ are projected onto the first two circular harmonics, yielding a complex dipole z_1 (polarity) and quadrupole z_2 (orientation, double-angle):

$$\boxed{z_n(p) = \sum_{k=1}^{8} h_k(p)\, e^{in\phi_k}, \qquad n \in \{1, 2\}.} \tag{7}$$

The second-harmonic magnitude $h_{2m}(p) = |z_2(p)|$ is exported separately. Image-border pixels (for which δ_k would step outside Ω) are zeroed. Only z_2 is used downstream; z_1 is exported for diagnostics. Because h_k is largest for offsets that cross an edge, $\theta_p = \frac{1}{2}\arg z_2(p)$ is the angle of the edge normal in the y-up frame. In image (y, x) order the normal is $(-\sin\theta_p, \cos\theta_p)$ and the tangent is $(\cos\theta_p, \sin\theta_p)$; all orientations below follow this convention.

2.2 L1: Cell-Level Orientation Binning

The padded image (section 2.1) is partitioned into overlapping $P \times P$ cells with stride $D = P - O$, indexed by c. Because $D < P$, a cell shares pixels with every cell up to $\lfloor (P-1)/D \rfloor = 2$ steps away in each direction, 24 neighbours in all. Inside cell c with pixel set $\mathcal{P}_c$, the per-pixel orientation estimate is $\theta_p = \frac{1}{2}\arg z_2(p)$. With K orientation bins of centres $\bar\theta_k = k\pi/K$ for $k = 0, \ldots, K-1$, the per-pixel von Mises weight is

$$v_k(\theta_p) = \exp\big(\kappa_{\mathrm{vm}}\,(\cos(2(\theta_p - \bar\theta_k)) - 1)\big), \tag{8}$$

with learnable concentration $\kappa_{\mathrm{vm}} > 0$ (softplus). The per-bin energy and sub-pixel anchors are

$$\boxed{\rho^{(k)}(c) = \sum_{p \in \mathcal{P}_c} |z_2(p)|\; v_k(\theta_p)\, \mathrm{ok}_p,} \tag{9}$$

$$a_x^{(k)}(c) = \frac{\sum_p |z_2(p)|\; v_k(\theta_p)\, x_p\, \mathrm{ok}_p}{\sum_p |z_2(p)|\; v_k(\theta_p)\, \mathrm{ok}_p + \varepsilon}, \tag{10}$$

and similarly $a_y^{(k)}(c)$, where $\mathrm{ok}_p \in \{0, 1\}$ excludes border pixels. Cells whose border-pixel fraction exceeds 0.2, or that extend past the unpadded image, are flagged as $\mathrm{ok}(c) = 0$: the seed sets their response to zero and the renderer deposits nothing from them. They still count as neighbours: the seed's facilitation and surround means (section 2.3) are normalised by fixed kernel sums, so a flagged cell enters them with value zero.

The output of L1 is the per-bin tensor $\rho^{(k)}(c) \in \mathbb{R}^{N \times K}$ together with the sub-pixel anchors $(a_x^{(k)}, a_y^{(k)})$ and the bin centres $\bar\theta_k$. L1 has no learned parameters of its own; κ_{vm} is stored with the seed's parameters, and L1 is recomputed at every training step from the live L0 output, so gradients reach κ_{vm} through the bin weights v_k.

2.3 Seed: Gain Control

The seed refines the L1 signal, converting the per-bin energies $\rho^{(k)}$ into a per-bin readout $\rho^{(k)}_{\text{out}}(c) \in [0,1]$, from which a per-cell scalar $\rho(c)$ and an orientation $\theta(c)$ are read. Each bin is facilitated by a co-circular association field along its tangent and divisively normalised against the same orientation in the surrounding cells.

The per-cell state the seed produces (a boundary strength $\rho(c)$, an orientation $\theta(c)$, and a sub-pixel anchor) is designed to be task-agnostic: a mid-level contour representation that downstream stages could consume directly for segmentation, grouping, or shape analysis. We use the term *seed* in this sense, as the distilled contour signal from which further processing proceeds. For the edge-detection task treated here that further processing is a single stage: the renderer (section 2.4) back-projects the seed onto the pixel grid. The seed's architecture does not depend on this choice of readout, but its parameters do: they are trained only through the renderer and the edge loss, and the per-cell exports are not on that path (see the cell-level export below), so their use as a general representation is untested here.

Per-bin Naka–Rushton squash: With $R^{(k)}(c) = \rho^{(k)}(c)\,\mathrm{ok}(c)$ and learnable half-saturation $\eta_z > 0$,

$$\rho^{(k)}_{\text{NR}}(c) = \frac{R^{(k)}(c)^2}{R^{(k)}(c)^2 + \eta_z^2 + \varepsilon}\,\mathrm{ok}(c). \tag{11}$$

Co-circular association field: For bin k let $\hat{t}_k = (\cos\bar{\theta}_k, \sin\bar{\theta}_k)$ and $\hat{n}_k = (-\sin\bar{\theta}_k, \cos\bar{\theta}_k)$ be the tangent and normal in (dy, dx) order. A cell offset $\delta \neq 0$ in the square neighbourhood $\mathcal{N}_f$ of radius r_f has tangent and normal coordinates $t_k(\delta) = \delta \cdot \hat{t}_k$ and $n_k(\delta) = \delta \cdot \hat{n}_k$, and makes the angle $\psi_k(\delta) = \arctan\big(n_k(\delta)/t_k(\delta)\big) \in (-\pi/2, \pi/2)$ with the tangent axis. Its weight

$$w_k(\delta) = \exp\left(-\frac{\|\delta\|^2}{2\sigma_f^2}\right)\exp\big(\kappa_a\big(\cos 2\psi_k(\delta) - 1\big)\big), \qquad \cos 2\psi_k(\delta) = \frac{t_k(\delta)^2 - n_k(\delta)^2}{\|\delta\|^2}, \tag{12}$$

separates reach (σ_f) from direction tuning (κ_a).

The neighbour at δ lies on the circle tangent to $\bar{\theta}_k$ at c with curvature $\kappa_k(\delta) = 2\,|n_k(\delta)| \,/\, \|\delta\|^2$, and the circle's tangent at $c{+}\delta$ is turned by $2\psi_k(\delta)$ [8]. Bin k reads the neighbour at the fractional bin position

$$q_k(\delta) = k + s_k(\delta)\,\frac{2\psi_k(\delta)\,K}{\pi}, \qquad s_k(\delta) = \frac{\sigma_\kappa^2}{\sigma_\kappa^2 + \kappa_k(\delta)^2}, \tag{13}$$

linearly interpolated between the two nearest bins (bin indices modulo K, $\{q\} = q - \lfloor q \rfloor$):

$$\tilde{\rho}^{(k)}_\delta(c) = \big(1 - \{q_k(\delta)\}\big)\,\rho^{(\lfloor q_k(\delta)\rfloor)}_{\text{NR}}(c+\delta) + \{q_k(\delta)\}\,\rho^{(\lfloor q_k(\delta)\rfloor+1)}_{\text{NR}}(c+\delta). \tag{14}$$

Turns implied by curvatures within the learned tolerance σ_κ (in 1/cell) are expected; sharper ones are read as straight continuation. At $\sigma_\kappa = 0$ the read is the same-bin collinear one.

Support is collected separately ahead of and behind the cell, each lobe normalised by its own weight,

$$f^{(k)}_\pm(c) = \frac{\sum_{\delta\in\mathcal{N}_f,\ \pm t_k(\delta)>0} w_k(\delta)\,\tilde{\rho}^{(k)}_\delta(c)}{\sum_{\delta\in\mathcal{N}_f,\ \pm t_k(\delta)>0} w_k(\delta) + \varepsilon}, \tag{15}$$

so a cell inside a long contour sees $f_+ \approx f_- \approx$ the contour's response; offsets with $t_k(\delta) = 0$ belong to neither lobe. The lobes give the mean collinear support and a gap bridge, their harmonic mean:

$$\rho^{(k)}_{\text{coll}}(c) = \tfrac{1}{2}\big(f^{(k)}_+(c) + f^{(k)}_-(c)\big), \qquad b^{(k)}(c) = \frac{2\,f^{(k)}_+(c)\,f^{(k)}_-(c)}{f^{(k)}_+(c) + f^{(k)}_-(c) + \varepsilon}. \tag{16}$$

The interpolation weights sum to one over the input bins, so the field is a bank of $2K \times K$ kernels of size $(2r_f+1)^2$, built from $(\sigma_f, \kappa_a, \sigma_\kappa)$ and applied as a single convolution with reflect padding.

Same-orientation Surround: Each bin is pooled over the neighbouring cells of the same bin with a Gaussian of width σ_S over the square neighbourhood $\mathcal{N}_S$ of radius r_S, the cell itself excluded:

$$S^{(k)}(c) = \frac{\sum_{\delta \in \mathcal{N}_S \setminus \{0\}} G_{\sigma_S}(\delta)\, \rho^{(k)}_{\mathrm{NR}}(c+\delta)}{\sum_{\delta \in \mathcal{N}_S \setminus \{0\}} G_{\sigma_S}(\delta)}, \qquad G_{\sigma_S}(\delta) = \exp\!\left(-\frac{\|\delta\|^2}{2\sigma_S^2}\right). \tag{17}$$

The pool is tuned exactly like the bins (through κ_{vm}), so a cell's own orientation never reaches another channel's pool, and the readout codes each cell relative to its same-orientation context. Only the cell is excluded, not its pixels: the 24 nearest cells overlap it (section 2.2), so S also reads part of the cell's own L1 input and acts as a near-field same-orientation pool rather than a surround outside the cell's footprint. The near offsets of the association field overlap the cell in the same way; section 4 gives the weight shares in the released model. G_{σ_S} is separable: it is applied as a depthwise row and column convolution (reflect padding) and the centre tap is subtracted.

Divisive Readout: Collinear support scales the cell's own response, and the bridge adds a term that is non-zero only where there is support on both sides:

$$e^{(k)}(c) = \rho^{(k)}_{\mathrm{NR}}(c)\left(\beta_{\mathrm{seed}} + \beta_{\mathrm{coll}}\, \rho^{(k)}_{\mathrm{coll}}(c)\right) + \beta_{\mathrm{gap}}\, b^{(k)}(c). \tag{18}$$

Because the facilitation is multiplicative it cannot create a response past a line end, and the bridge fills gaps without extending ends. The excitation is divisively normalised against the surround through a Michaelis–Menten-style readout [1, 3]:

$$\boxed{\rho^{(k)}_{\mathrm{out}}(c) = \frac{\left(e^{(k)}(c)\right)^2}{\left(e^{(k)}(c)\right)^2 + \eta_r^2 + \lambda\left(S^{(k)}(c)\right)^2 + \varepsilon},} \tag{19}$$

with learnable gains $(\beta_{\mathrm{seed}}, \beta_{\mathrm{coll}}, \beta_{\mathrm{gap}})$, readout half-saturation η_r and surround weight λ, all positive via softplus. The squared e on numerator and denominator gives a Naka–Rushton-shaped transfer with bounded range, while λS^2 raises the half-saturation point where the neighbouring cells carry the same orientation.

Cell-level Export: Three quantities are exported per cell. The scalar boundary strength $\rho(c) \in [0,1]$ is the maximum over bins, returned through a straight-through estimator so the forward pass is hard while the backward pass is smooth:

$$\rho(c) = \max_k \rho^{(k)}_{\mathrm{out}}(c), \qquad \tilde{\rho}(c) = \sum_k \mathrm{softmax}_\tau\!\left(\rho^{(\cdot)}_{\mathrm{out}}(c)\right)_k \rho^{(k)}_{\mathrm{out}}(c), \tag{20}$$

with the standard STE construction $\rho(c) - \tilde{\rho}(c)_{\mathrm{detach}} + \tilde{\rho}(c)$ on the forward graph (temperature τ fixed). The cell orientation is a double-angle weighted blend across bins,

$$\theta(c) = \tfrac{1}{2}\,\mathrm{atan2}\!\Big(\sum_k \rho^{(k)}_{\mathrm{out}}(c)\, \sin 2\bar{\theta}_k,\ \sum_k \rho^{(k)}_{\mathrm{out}}(c)\, \cos 2\bar{\theta}_k + \varepsilon\Big), \tag{21}$$

and the cell anchor is taken at the dominant bin $k^\star(c) = \arg\max_k \rho^{(k)}_{\mathrm{out}}(c)$:

$$\left(a^\star_x(c),\, a^\star_y(c)\right) = \left(a^{(k^\star)}_x(c),\, a^{(k^\star)}_y(c)\right). \tag{22}$$

The renderer consumes the full per-bin tensor $\rho_{\text{out}}^{(k)} \in \mathbb{R}^{N\times K}$, the per-bin anchors $(a_x^{(k)}, a_y^{(k)})$ and the bin centres $\bar{\theta}_k$, not $\rho(c)$, so that it can deposit each bin separately and aggregate with a noisy-OR. The cell-level quantities $\rho(c)$, $\theta(c)$ and $a^\star(c)$ are exports of the mid-level representation and are not on the training-loss path; in particular the STE temperature τ does not affect training.

2.4 Renderer: Ramp-filtered Back-projection

The renderer deposits each active (cell, bin) pair onto the pixel grid as a thin curved stroke shaped by two learned 1D kernels, then aggregates contributions via a noisy-OR. The construction borrows the geometry of filtered back-projection [12]; each pair contributes mass along an oriented line through its anchor, and the radial filter is constrained to the windowed-ramp inverse-Radon family. This is not a filtered back-projection in the analytic sense: L0+L1 produce a local orientation-energy field rather than line integrals, the longitudinal profile $h_{\parallel}$ is a learned finite-support stroke rather than a ray delta, and aggregation across pairs is multiplicative (noisy-OR) rather than additive. The renderer thus exploits FBP's back-projection topology while leaving the inversion identity behind.

Per-bin Features and Correction MLP: For each (cell, bin) pair a 5-dimensional feature vector is computed from $\rho_{\text{out}}^{(k)}$ and its 3×3 neighbourhood:

$$F_c^{(k)} = \big[\, \rho_{\text{out}}^{(k)}(c),\ \mu_{\text{coll}}^{(k)}(c),\ \mu_t^{(k)}(c),\ \textstyle\sum_{j\neq k} \rho_{\text{out}}^{(j)}(c),\ \mu_n^{(k)}(c) \,\big] \in \mathbb{R}^5, \tag{23}$$

where $\mu_{\text{coll}}^{(k)}$ is a position-weighted collinear support, and $\mu_t^{(k)}, \mu_n^{(k)}$ are signed tangent and normal asymmetries of $\rho_{\text{out}}^{(k)}$ across the 3×3 neighbourhood (concrete definitions in appendix A). The features are computed on a detached copy of ρ_{out}, with zeros outside the cell grid and at border cells. A small shared MLP $\Phi : \mathbb{R}^5 \to \mathbb{R}^4$ with one hidden layer of width 12 emits four raw outputs which are bounded by tanh and softplus:

$$\begin{aligned}
\kappa_c^{(k)} &= \kappa_{\max}\, \tanh\big(\Phi_1(F_c^{(k)})\big), \\
\xi_c^{(k)} &= \xi_{\max}\, \tanh\big(\Phi_2(F_c^{(k)})\big), \\
\delta_{n_c}^{(k)} &= \delta_{n,\max}\, \tanh\big(\Phi_3(F_c^{(k)})\big), \\
\alpha_c^{(k)} &= \exp\big(\alpha_{\text{range}}\, \tanh\big(\Phi_4(F_c^{(k)})\big)\big),
\end{aligned} \tag{24}$$

where $\kappa_{\max}, \xi_{\max}, \delta_{n,\max}, \alpha_{\text{range}}$ are learnable positive bounds. The four scalars are a curvature, a tangent-direction shift of the stroke vertex, a normal-direction anchor correction, and a multiplicative amplitude $\alpha \in [e^{-\alpha_{\text{range}}}, e^{+\alpha_{\text{range}}}]$. At initialisation Φ outputs nearly zero, so $\kappa, \xi, \delta_n \approx 0$ and $\alpha \approx 1$, recovering an unmodulated straight stroke.

Sparsity Gate: Each cell's bins compete via a winner-decay gate

$$g^{(k)}(c) = \exp\left(\frac{\rho_{\text{out}}^{(k)}(c) - \max_j \rho_{\text{out}}^{(j)}(c)}{T_{\text{gate}}} \right) \text{ok}(c), \tag{25}$$

with learnable temperature $T_{\text{gate}} > 0$. The dominant bin always satisfies $g = 1$; lagging bins decay exponentially in the ρ-gap. Co-equal maxima (junctions) both fire at full gain. Pairs with $g < 10^{-3}$ or $\rho_{\text{out}}^{(k)} < 10^{-4}$ contribute nothing. On CPU and CUDA they are dropped from the active set, so the MLP and synthesis run only on surviving pairs; on Apple GPUs (MPS) every pair is kept with zero amplitude, so that tensor shapes depend only on the image size. Both give the same map.

Anchor correction. The bin anchor $(a_x^{(k)}, a_y^{(k)})$ from eq. (10) is shifted along the bin normal by δ_n:

$$\tilde{a}_c^{(k)} = \big(a_x^{(k)}(c) + \delta_{n_c}^{(k)} \cos\bar{\theta}_k,\ a_y^{(k)}(c) - \delta_{n_c}^{(k)} \sin\bar{\theta}_k\big). \tag{26}$$

Curved Stroke Synthesis: For each pixel p in the deposit window of $\tilde{a}_c^{(k)}$, let $\Delta = p - \tilde{a}_c^{(k)}$. Project onto the bin tangent and normal,

$$s = \Delta_y \cos\bar{\theta}_k + \Delta_x \sin\bar{\theta}_k, \qquad n = -\Delta_y \sin\bar{\theta}_k + \Delta_x \cos\bar{\theta}_k, \tag{27}$$

shift the tangent coordinate by ξ and bend the normal coordinate by the parabolic curvature term,

$$\tilde{s} = s - \xi_c^{(k)}, \qquad n_c = n - \tfrac{1}{2}\kappa_c^{(k)}\, \tilde{s}^2, \tag{28}$$

and synthesise the (non-negative) stroke profile by a product of learned 1D kernels:

$$f_c^{(k)}(p) = \mathrm{ReLU}\big(h_\perp(n_c)\, h_\parallel(\tilde{s})\big). \tag{29}$$

Both kernels are evaluated at continuous query positions by linear interpolation between integer taps.

Learned 1D Kernels: The radial filter $h_\perp$ is parameterised as a windowed ramp from the inverse-Radon family,

$$h_\perp[n] = \mathrm{IFFT}\big(|\omega|\ \mathcal{W}(\omega)\big)[n], \qquad n \in \{-H_w, \ldots, H_w\}, \tag{30}$$

where $\mathcal{W}(\omega) \geq 0$ is a monotonically decreasing window in $|\omega|$, parameterised by $H_w + 1$ logits $(\nu_0, \ldots, \nu_{H_w})$ via $\mathcal{W}(0) = \mathrm{softplus}(\nu_0)$, $\mathcal{W}(\omega_{m+1}) = \mathcal{W}(\omega_m)\, \sigma(\nu_{m+1})$. This constrains $h_\perp$ to the shape of an FBP ramp filter and initialises it to a Hann window of cutoff ω_c; the resulting profile is the FBP-shaped piece of the renderer.

The longitudinal profile $h_\parallel$ is an even, monotonically decreasing function from a positive peak, parameterised by H_w decay-ratio logits $(\zeta_1, \ldots, \zeta_{H_w})$ and one peak parameter h_0:

$$h_\parallel[m] = h_0 \prod_{j=1}^{|m|} \sigma(\zeta_j), \qquad h_0 = \mathrm{softplus}(h_{0,\mathrm{raw}}), \tag{31}$$

initialised to a Gaussian of width $\sigma_{\parallel,0}$.

Per-pair Claim and Noisy-OR Aggregation: The per-pair pixel claim is

$$\mathrm{claim}_c^{(k)}(p) = \alpha_c^{(k)}\, g^{(k)}(c)\, \rho_{\mathrm{out}}^{(k)}(c)\, f_c^{(k)}(p), \tag{32}$$

clamped into $[0, 1 - \epsilon_c]$ for numerical stability, and the global boundary map is the noisy-OR over all active pairs:

$$\boxed{\hat{B}(p) = 1 - \prod_{(c,k)\in\mathcal{A}} \big(1 - \mathrm{claim}_c^{(k)}(p)\big).} \tag{33}$$

In log space, $\log(1 - \hat{B}(p)) = \sum_{(c,k)} \log(1 - \mathrm{claim}_c^{(k)}(p))$, which is computed by scatter-add on the pixel-flat index and stably back-propagates through both the synthesis and the learned kernels. The anchors and the MLP features are treated as constants in the backward pass, so the renderer passes gradient to the seed and earlier stages only through $\rho_{\mathrm{out}}^{(k)}$, in the claim and in the gate.

Inference: At inference, $\hat{B}$ is thinned by ridge non-maximum suppression along the claim-weighted dominant orientation $\theta^\star(p)$ (the circular mean of $\bar{\theta}_k$ weighted by per-pair claims): a pixel is kept if it is no smaller than its bilinearly sampled neighbours one pixel either way along the normal. The thinned map is thresholded at $t_{\text{edge}} = 1/2$; for benchmarking the threshold is swept instead (section 4).

3 Training

The loss is HED-style class-balanced binary cross-entropy [14]. Let $y(p) \in [0, 1]$ be the ground-truth target and $\hat{B}(p)$ the prediction, clamped to $[10^{-6}, 1 - 10^{-6}]$. The loss is taken over the original image Ω_0, which has n_+ positive pixels ($y(p) \geq \frac{1}{2}$) and $n_- = |\Omega_0| - n_+$ negative ones:

$$\mathcal{L} = -\frac{1}{|\Omega_0|} \sum_{p \in \Omega_0} \Big[\frac{n_-}{|\Omega_0|}\, y(p) \log \hat{B}(p) + \frac{n_+}{|\Omega_0|}\, (1 - y(p)) \log(1 - \hat{B}(p)) \Big], \tag{34}$$

computed per image and averaged over the batch. Gradients flow through all four stages: through the renderer's MLP, correction bounds, gate temperature and 1D kernels; through the seed's eleven scalars; through the L1 von Mises weights via κ_{vm}; and through the L0 metric $(w_\ell, s_1, s_2, \theta_{\text{chr}})$, notch (ω_n, σ_n, d) and gain control (λ_0, σ_r). Because every L0 parameter is learned, L0 and L1 are recomputed at every step; the training cache holds only the padded RGB image, the ground truth and the border mask.

We train on the 300-image BRIND train split [9][1], the BSDS500 images [11] re-annotated for reflectance, illumination, normal and depth discontinuities and merged into one edge map per image. We use Adam [13] with a learning rate of 5×10^{-2} annealed on a cosine schedule to 5×10^{-3}, gradient clipping at $\|\nabla\|_2 \leq 1$, batch size 4, for 50 epochs.

4 Results

Table 1 reports the released model on the 200-image BRIND test set. Evaluation follows the Berkeley boundary benchmark [11]: at each of 99 thresholds the prediction is binarised, thinned to one-pixel width and matched one-to-one against the ground truth within 0.0075 of the image diagonal ($\approx$ 4.3 px on 481 $\times$ 321 images). ODS is the F-measure at the best dataset-wide threshold, OIS the F-measure of the match counts pooled with each image at its own best threshold, and AP the area under the precision–recall curve. Two outputs are scored: the ridge-NMS map of section 2.4 and the soft map $\hat{B}$.

Table 1: BRIND test set (200 images), trained on the 300-image BRIND train split. Precision and recall are taken at the ODS threshold (0.74 for both maps).

Output	ODS	Precision	Recall	OIS	AP
Ridge-NMS map	0.7072	0.6745	0.7432	0.7318	0.7549
Soft map $\hat{B}$	0.7043	0.6777	0.7331	0.7274	0.6524

Three learned properties change how the corresponding stages should be read. The notch moved away from its JPEG initialisation to $\omega_n = 0.41$, $\sigma_n = 0.18$, $d = 0.83$; at the five sampled frequencies $0, 1/9, \ldots, 4/9$ cycles/px its gain is $0.94, 0.79, 0.52, 0.25, 0.19$, so it acts as a broad high-frequency cut rather than a notch. The curvature tolerance fell to $\sigma_\kappa = 0.002$, which shifts the co-circular read of eq. (13) by at most 0.01 of a bin: in the released model the association field is effectively collinear. Finally, the learned widths $\sigma_S = 1.80$ and $\sigma_f = 1.86$ cells keep both

[1] https://github.com/xavysp/BRIND



(a) Input image I.



(b) Detected edges: ridge-NMS of $\hat{B}$, thresholded at $t_{\text{edge}} = 1/2$.

Figure 1: Qualitative result on one natural image from outside the BRIND splits, with the released model. Left: input. Right: HaCI edge map at inference.

seed pools close to the cell: 69% of the surround weight, and 65–73% of each facilitation lobe's weight depending on the bin, falls on the 24 cells that share pixels with it.

5 Qualitative Example

HaCI applied to a single natural image is shown in fig. 1. The input (fig. 1a) is a 1280×961 full-colour photograph of zebras on dry grassland, from outside the BRIND splits, with luminance and chrominance contrasts that L0 routes through the learned spectral metric. The output (fig. 1b) is the inference output of section 2.4: $\hat{B}$ thinned by ridge non-maximum suppression along the claim-weighted orientation $\theta^\star(p)$ and thresholded at $t_{\text{edge}} = 1/2$. White pixels mark the surviving one-pixel-wide contours.

6 Parameter Inventory

The pipeline has 159 learnable parameters (table 3) and a handful of fixed structural constants (table 2). The renderer's correction MLP accounts for 124 of the learned parameters; everything else is interpretable scalars (gains, half-saturation constants, anisotropy widths) or small monotone-parameterised filter taps.

7 Discussion

HaCI compresses a classical contour-integration computation into a fully differentiable graph of 159 scalars. The choice of what to expose to data and what to fix is rooted in the strict interpretation of cortical mechanisms. The renderer's $5 \to 12 \to 4$ correction head is the only learned component that is neither a named scalar nor a constrained filter; it is there to handle off-anchor edges, sub-pixel curvature and bin selectivity at junctions. With its output held at zero ($\kappa = \xi = \delta_n = 0$, $\alpha = 1$) the renderer reduces to a straight-stroke depositor; how much the head adds over that is left to the ablations below. The kernels $h_\perp, h_\parallel$ are parameterised so that every point in parameter space is a valid windowed-ramp radial filter and a positive monotone-decay longitudinal profile, respectively. This keeps the synthesis path interpretable as a ramp-filtered stroke deposit, while leaving the kernel shape open to gradient descent against the boundary objective.

Table 2: Fixed structural and numerical constants.

Block	Symbol	Value
L0 luminance half-saturation	η_{lum}	0.01
L0 chrominance half-saturation	η_{chr}	0.01
L0 notch half-width	H_N	4
L0 gain-control pool radius	r_Q	16 px
L1 cell size	P	5 px
L1 cell overlap	O	3 px (stride $D = 2$)
L1 border-cell fraction	—	0.2
L1/Seed orientation bins	K	8
Seed association-field radius	r_f	5 cells
Seed surround radius	r_S	10 cells
Seed STE temperature	τ	0.1
Renderer deposit half-width	H_w	$\lceil 2D \rceil = 4$ px
Inference edge threshold	t_{edge}	0.5

This report fixes the equations and reports results on BRIND (section 4); ablations and comparison against deep baselines [14, 15, 16, 17, 18, 19] are deferred to a forthcoming full paper.

Code availability

The reference implementation is available at https://github.com/RenderBear/haci.

A Per-bin Renderer Features

The five features in eq. (23) are computed from the 3×3 neighbourhood $\mathcal{N}_3(c)$ of cell c (with $\delta \in \mathcal{N}_3 \setminus \{0\}$) as follows. Let $t_k(\delta) = \delta_y \cos\bar{\theta}_k + \delta_x \sin\bar{\theta}_k$ and $n_k(\delta) = -\delta_y \sin\bar{\theta}_k + \delta_x \cos\bar{\theta}_k$ be the tangent and normal projections of the offset in bin k, and $\text{pos}_t^{(k)}(\delta) = t_k(\delta)^2 / \|\delta\|^2$. Then

$$\mu_{\text{coll}}^{(k)}(c) = \frac{\sum_\delta \rho_{\text{out}}^{(k)}(c+\delta)\, \text{pos}_t^{(k)}(\delta)}{\sum_\delta \text{pos}_t^{(k)}(\delta) + \varepsilon'}, \tag{35}$$

$$\mu_t^{(k)}(c) = \frac{\sum_\delta \rho_{\text{out}}^{(k)}(c+\delta)\, t_k(\delta)}{\sum_\delta \rho_{\text{out}}^{(k)}(c+\delta)\ |t_k(\delta)| + \varepsilon'}, \tag{36}$$

$$\mu_n^{(k)}(c) = \frac{\sum_\delta \rho_{\text{out}}^{(k)}(c+\delta)\, n_k(\delta)}{\sum_\delta \rho_{\text{out}}^{(k)}(c+\delta)\ |n_k(\delta)| + \varepsilon'}, \tag{37}$$

with a soft floor $\varepsilon' = 5 \times 10^{-2}$ on the denominator. $\mu_{\text{coll}}^{(k)}$ summarises collinear support along the bin tangent; $\mu_t^{(k)} \in [-1, 1]$ is a signed tangent asymmetry that signals where, along the tangent, the local mass is concentrated, and is meant as the cue for the tangent shift ξ; $\mu_n^{(k)} \in [-1, 1]$ is the analogous normal asymmetry, meant as the cue for δ_n. Nothing enforces this pairing: Φ is a dense MLP, so every feature reaches all four outputs.

Table 3: Learned parameters. Positive scalars are reparameterised through softplus unless noted: ω_n and d use a sigmoid, $\sigma_f = 1 + \mathrm{softplus}(\cdot)$, and σ_S is clamped below at 0.3. Angles are unbounded.

Block	Symbol(s)	Init	#params
L0 metric gains	w_ℓ, s_1, s_2	1.0	3
L0 chrominance angle	θ_{chr}	0	1
L0 notch	ω_n, σ_n, d	$1/8, 1/32, 0.8$	3
L0 gain-control pool	λ_0, σ_r	$10^{-5}, 2.0$	2
L0 subtotal			**9**
Seed von Mises concentration	κ_{vm}	2.0	1
Seed NR half-saturation	η_z	10.0	1
Seed channel gain	β_{seed}	0.5	1
Seed collinear gain	β_{coll}	0.5	1
Seed gap-bridge gain	β_{gap}	0.5	1
Seed readout half-saturation	η_r	0.3	1
Seed surround weight	λ	0.5	1
Seed association reach	σ_f (≥ 1)	2.0	1
Seed association direction tuning	κ_a	8.0	1
Seed curvature tolerance	σ_κ	0.01	1
Seed surround width	σ_S (≥ 0.3)	1.0	1
Seed subtotal			**11**
Renderer correction MLP $5 \to 12 \to 4$	Φ	near-zero output	124
Renderer correction bounds	$\kappa_{\max}, \xi_{\max}, \delta_{n,\max}, \alpha_{\mathrm{range}}$	$0.1, 1, 1, 0.5$	4
Renderer gate temperature	T_{gate}	0.08	1
Renderer radial filter	$\{\nu_m\}_{m=0}^{H_w}$	Hann $\omega_c = 0.5$	5
Renderer longitudinal filter	$\{\zeta_j\}_{j=1}^{H_w}, h_0$	Gaussian $\sigma_{\parallel,0} = 2$	5
Renderer subtotal			**139**
Total			**159**

B Renderer parameter count

The correction MLP has $5 \cdot 12 + 12 = 72$ parameters in the input layer and $12 \cdot 4 + 4 = 52$ in the output layer, for 124 total. The kernel radial logits $\{\nu_m\}$ have $H_w + 1 = 5$ parameters; the longitudinal logits $\{\zeta_j\}$ have $H_w = 4$ parameters; the longitudinal peak adds 1. The four correction bounds $(\kappa_{\max}, \xi_{\max}, \delta_{n,\max}, \alpha_{\mathrm{range}})$ and the gate temperature T_{gate} contribute 5 further scalars. Total: $124 + 4 + 1 + 5 + 4 + 1 = 139$.

References

[1] K. I. Naka, W. A. H. Rushton. S-potentials from colour units in the retina of fish (Cyprinidae). *Journal of Physiology*, 185(3):536–555, 1966.

[2] D. H. Hubel, T. N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *Journal of Physiology*, 160(1):106–154, 1962.

[3] D. J. Heeger. Normalization of cell responses in cat striate cortex. *Visual Neuroscience*, 9(2):181–197, 1992.

[4] J. R. Cavanaugh, W. Bair, J. A. Movshon. Nature and interaction of signals from the receptive field center and surround in macaque V1 neurons. *Journal of Neurophysiology*, 88(5):2530–2546, 2002.

[5] M. K. Kapadia, M. Ito, C. D. Gilbert, G. Westheimer. Improvement in visual sensitivity by changes in local context: parallel studies in human observers and in V1 of alert monkeys. *Neuron*, 15(4):843–856, 1995.

[6] D. J. Field, A. Hayes, R. F. Hess. Contour integration by the human visual system: Evidence for a local "association field". *Vision Research*, 33(2):173–193, 1993.

[7] H. Zhou, H. S. Friedman, R. von der Heydt. Coding of border ownership in monkey visual cortex. *Journal of Neuroscience*, 20(17):6594–6611, 2000.

[8] P. Parent, S. W. Zucker. Trace inference, curvature consistency, and curve detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(8):823–839, 1989.

[9] M. Pu, Y. Huang, Q. Guan, H. Ling. RINDNet: Edge Detection for Discontinuity in Reflectance, Illumination, Normal and Depth. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[10] J. Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.

[11] P. Arbeláez, M. Maire, C. Fowlkes, J. Malik. Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916, 2011.

[12] A. C. Kak, M. Slaney. *Principles of Computerized Tomographic Imaging*. SIAM Classics in Applied Mathematics 33, 2001.

[13] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[14] S. Xie, Z. Tu. Holistically-Nested Edge Detection. In *IEEE International Conference on Computer Vision (ICCV)*, 2015.

[15] Y. Liu, M.-M. Cheng, X. Hu, K. Wang, X. Bai. Richer Convolutional Features for Edge Detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[16] J. He, S. Zhang, M. Yang, Y. Shan, T. Huang. BDCN: Bi-Directional Cascade Network for Perceptual Edge Detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[17] M. Pu, Y. Huang, Y. Liu, Q. Guan, H. Ling. EDTER: Edge Detection with Transformer. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[18] X. Soria, A. Sappa, P. Humanante, A. Akbarinia. Dense Extreme Inception Network for Edge Detection. *Pattern Recognition*, 139:109461, 2023.

[19] Y. Ye, K. Xu, Y. Huang, R. Yi, Z. Cai. DiffusionEdge: Diffusion Probabilistic Model for Crisp Edge Detection. In *AAAI Conference on Artificial Intelligence*, 2024.